SQL Server اليوم 9 · الأسبوع 2 · ساعتان · دورة أساسيات البرمجة

تعديل البيانات والتجميع وربط #C بـ SQL Server

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- يعدّل البيانات بـ UPDATE ويحذفها بـ DELETE مع WHERE دائمًا.
- يحسب الإحصائيات بـ COUNT وAVG وMAX وMIN وGROUP BY.
- يتصل بقاعدة البيانات من برنامج #C باستخدام Microsoft.Data.SqlClient.
- يقرأ الصفوف بـ SqlDataReader ويضيف صفوفًا باستعلام ذي باراميترات.

توزيع وقت الحصة

0:00 - 0:10	مراجعة SELECT وWHERE
0:10 - 0:35	UPDATE وDELETE ودوال التجميع
0:35 - 1:05	ربط #C بـ SQL Server: القراءة والإضافة
1:05 - 1:15	استراحة
1:15 - 1:40	تمارين: برنامج #C متصل بالقاعدة
1:40 - 2:00	العمل على المشروع النهائي بإشراف المدرب

شرح الدرس

1. التعديل والحذف

```sql
UPDATE Students SET Grade = 82 WHERE Id = 2;

UPDATE Students SET Course = N'C#', Age = 24 WHERE FullName = N'علي';

DELETE FROM Students WHERE Id = 6;
```

ماذا يفعل هذا الكود؟ أوامر لتعديل بيانات موجودة (UPDATE) وحذف صف (DELETE) من جدول Students. عند التشغيل تظهر رسالة بعدد الصفوف التي تأثرت مثل (1 row affected).

```
UPDATE Students SET Grade = 82
WHERE Id = 2;
```

الأمر UPDATE يعدّل بيانات موجودة، و SET تحدد العمود وقيمته الجديدة، و WHERE تحدد أي صف سيتعدّل (الطالب صاحب Id = 2). تحذير: لو نسيت WHERE ستتغير درجة كل الطلاب في الجدول!

```
UPDATE Students SET Course =
N'C#', Age = 24 WHERE FullName =
N'علي';
```

يمكن تعديل أكثر من عمود في أمر واحد بفصلها بفاصلة بعد SET. هنا نغيّر دورة وعمر الطالب الذي اسمه علي، والبادئة N ضرورية لأن الاسم عربي.

`DELETE FROM Students WHERE Id = 6;`	الأمر DELETE FROM يحذف صفوفًا من الجدول، و WHERE تحدد الطالب صاحب Id = 6 فقط. تحذير مهم: DELETE بدون WHERE يحذف كل الصفوف، ولا يوجد زر تراجع، لذلك تأكد دائمًا من الشرط.

تحذير مهم: UPDATE أو DELETE بدون WHERE يغيّر أو يحذف كل صفوف الجدول! اكتب SELECT بنفس الشرط أولًا لتتأكد.

2. دوال التجميع

```SQL
SELECT COUNT(*) AS StudentsCount FROM Students;
SELECT AVG(Grade) AS Average, MAX(Grade) AS Highest, MIN(Grade) AS Lowest FROM Students;

-- عدد الطلاب ومتوسطهم لكل دورة
SELECT Course, COUNT(*) AS Total, AVG(Grade) AS Avg
FROM Students
GROUP BY Course;
```

ماذا يفعل هذا الكود؟ يستخدم الدوال التجميعية لحساب إحصائيات عن الطلاب: العدد، والمتوسط، وأعلى وأدنى درجة، ثم نفس الإحصائيات لكل دورة على حدة باستخدام GROUP BY.

`SELECT COUNT(*) AS StudentsCount FROM Students;`	الدالة COUNT(*) تعدّ صفوف الجدول وتعيد رقمًا واحدًا هو عدد الطلاب. الكلمة AS تعطي عمود النتيجة اسمًا مستعارًا (StudentsCount) ليظهر بعنوان واضح بدل عمود بلا اسم.
`SELECT AVG(Grade) AS Average, MAX(Grade) AS Highest, MIN(Grade) AS Lowest FROM Students;`	الدالة AVG تحسب متوسط الدرجات، و MAX تعيد أعلى درجة، و MIN أدنى درجة. النتيجة صف واحد فيه ثلاثة أعمدة، ولكل عمود اسم أعطيناه بـ AS.
`-- عدد الطلاب ومتوسطهم لكل دورة`	تعليق يبدأ بـ -- لشرح الاستعلام التالي، ولا ينفذه SQL Server.
`SELECT Course, COUNT(*) AS Total, AVG(Grade) AS Avg`	نختار اسم الدورة، وعدد الطلاب (Total)، ومتوسط الدرجات (Avg). هذا الاستعلام مكتوب على عدة أسطر لسهولة القراءة، وهو أمر واحد ينتهي عند الفاصلة المنقوطة.
`FROM Students`	نأخذ البيانات من جدول Students.
`GROUP BY Course;`	الكلمة GROUP BY تقسّم الطلاب إلى مجموعات حسب الدورة، ثم تُحسب COUNT و AVG لكل مجموعة على حدة. النتيجة صف لكل دورة. قاعدة: أي عمود عادي في SELECT (مثل Course) يجب أن يكون مذكورًا في GROUP BY.

3. ربط C# بـ SQL Server

1) في Visual Studio: Project ← Manage NuGet Packages ← ابحث عن `Microsoft.Data.SqlClient` وثبّته.
2) جهّز **نص الاتصال** Connection String.
3) افتح الاتصال، نفذ الأمر، اقرأ النتائج.

C#

```
using Microsoft.Data.SqlClient;

string cs =
@"Server=localhost\SQLEXPRESS;Database=TrainingDB;Trusted_Connection=True;TrustServerCerti
ficate=True;";

using var conn = new SqlConnection(cs);
conn.Open();

var cmd = new SqlCommand("SELECT Id, FullName, Grade FROM Students ORDER BY Grade DESC",
conn);
using var reader = cmd.ExecuteReader();

while (reader.Read())
{
    Console.WriteLine($"{reader["Id"]}. {reader["FullName"]} - {reader["Grade"]}");
}
```

ماذا يفعل هذا الكود؟ برنامج C# يتصل بقاعدة البيانات TrainingDB ويقرأ الطلاب مرتبين حسب الدرجة من الأعلى، ثم يطبع كل طالب في سطر بالشكل: الرقم. الاسم - الدرجة.

`using Microsoft.Data.SqlClient;`	توجيه using في أول الملف يستورد مكتبة Microsoft.Data.SqlClient حتى نستخدم أصنافها مثل SqlConnection و SqlCommand بأسمائها المختصرة. يجب أولًا إضافة حزمة NuGet بنفس الاسم إلى المشروع.
`string cs = @"Server=localhost\SQLEXPRESS;Database=TrainingDB;Trusted_Connection=True;TrustServerCertificate=True;";`	هذا نص الاتصال (connection string) ويخبر البرنامج كيف يصل لقاعدة البيانات: Server هو اسم الخادم (localhost\SQLEXPRESS أي نسخة SQLEXPRESS على جهازك)، و Database اسم القاعدة، و Trusted_Connection=True يعني الدخول بحساب Windows بلا كلمة مرور، و TrustServerCertificate=True يقبل شهادة الأمان المحلية للخادم دون تحقق. علامة @ قبل النص تجعل \ حرفًا عاديًا وليس بداية رمز خاص.
`using var conn = new SqlConnection(cs);`	ننشئ كائن اتصال SqlConnection باستخدام نص الاتصال. كلمة using var (تختلف عن توجيه using في أول الملف) تضمن إغلاق الاتصال وتحرير موارده تلقائيًا عند انتهاء البرنامج أو الجزء الذي هو فيه، حتى لو حدث خطأ.

`conn.Open();`	الدالة Open تفتح الاتصال فعليًا بالخادم. لا يمكن تنفيذ أي أمر قبل فتح الاتصال، وإن كانت بيانات الاتصال خاطئة سيظهر الخطأ هنا.
`var cmd = new SqlCommand("SELECT Id, FullName, Grade FROM Students ORDER BY Grade DESC", conn);`	الكائن SqlCommand يحمل أمر SQL الذي نريد تنفيذه (هنا استعلام SELECT مرتّب تنازليًا بالدرجة) والاتصال الذي سيُنفّذ عليه.
`using var reader = cmd.ExecuteReader();`	الدالة ExecuteReader تنفّذ الاستعلام وتعيد قارئًا (SqlDataReader) نقرأ به النتائج صفًا صفًا. نستخدم using var لإغلاق القارئ تلقائيًا عند الانتهاء.
`while (reader.Read())`	الدالة Read تنتقل إلى الصف التالي وتعيد true إن وُجد صف، و false عندما تنتهي الصفوف. لذلك نضعها في حلقة while لتكرار القراءة حتى آخر صف.
`{`	القوس المعقوص { يبدأ جسم الحلقة، أي الأوامر التي تتكرر لكل صف، والقوس } ينهيه.
`Console.WriteLine($" {reader["Id"]}. {reader["FullName"]} - {reader["Grade"]}");`	نطبع الصف الحالي. reader["Id"] يجلب قيمة العمود Id من الصف الحالي بكتابة اسم العمود، وكذلك FullName و Grade. علامة $ قبل النص تسمح بوضع القيم داخل { } مباشرة في النص.

4. الإضافة باستخدام البارامترات

لا تدمج مدخلات المستخدم مباشرة في نص الاستعلام، بل استخدم **بارامترات** مثل `@name` . هذا يحمي من ثغرة SQL Injection ويتعامل مع العربية بشكل صحيح.

C#

```
Console.Write("الاسم: ");
string name = Console.ReadLine();
Console.Write("الدرجة: ");
decimal grade = decimal.Parse(Console.ReadLine());

using var conn = new SqlConnection(cs);
conn.Open();
var cmd = new SqlCommand(
    "INSERT INTO Students (FullName, Grade) VALUES (@name, @grade)", conn);
cmd.Parameters.AddWithValue("@name", name);
cmd.Parameters.AddWithValue("@grade", grade);

int rows = cmd.ExecuteNonQuery();
Console.WriteLine($"تمت إضافة {rows} صف");
```

ماذا يفعل هذا الكود؟ برنامج يطلب من المستخدم اسم طالب ودرجته، ثم يضيفه إلى جدول Students بأمان باستخدام المعاملات (parameters)، ويطبع عدد الصفوف التي أضيفت: تمت إضافة 1 صف.

`Console.Write("الاسم: ");`	نطبع سؤالًا للمستخدم. Write تبقي المؤشر على نفس السطر ليكتب المستخدم الإجابة بجانب السؤال.
`string name = Console.ReadLine();`	ReadLine تنتظر المستخدم حتى يكتب نصًا ويضغط Enter، ثم نحفظ ما كتبه في المتغير name.
`Console.Write("الدرجة: ");`	نطلب من المستخدم إدخال الدرجة.
`decimal grade = decimal.Parse(Console.ReadLine());`	ما يكتبه المستخدم يصل دائمًا كنص، لذلك نحوّله إلى رقم عشري بـ decimal.Parse. إن كتب المستخدم شيئًا ليس رقمًا سيتوقف البرنامج بخطأ.
`using var conn = new SqlConnection(cs);`	ننشئ اتصالًا بقاعدة البيانات باستخدام نص الاتصال cs المعرّف في المثال السابق، و using var تغلقه تلقائيًا عند الانتهاء.
`conn.Open();`	نفتح الاتصال بالخادم قبل تنفيذ أي أمر.
`var cmd = new SqlCommand(`	ننشئ أمر SqlCommand، وقد كتبناه على سطرين لأن النص طويل.
`"INSERT INTO Students (FullName, Grade) VALUES (@name, @grade)", conn);`	هذا أمر الإضافة، لكن بدل وضع القيم مباشرة كتبنا @name و @grade، وهي معاملات (parameters): أماكن محجوزة ستُملأ بالقيم لاحقًا بشكل منفصل عن نص الأمر.
`cmd.Parameters.AddWithValue("@name", name);`	AddWithValue تربط المعامل @name بقيمة المتغير name. القيمة تُرسل للخادم كبيانات فقط وليست كجزء من أمر SQL، فلو كتب المستخدم نصًا خبيثًا لن يُنفّذ كأمر، وهذا يحمينا من هجوم SQL Injection (حقن أوامر ضارة).
`cmd.Parameters.AddWithValue("@grade", grade);`	نربط المعامل @grade بقيمة المتغير grade بنفس الطريقة الآمنة. لا تبنِ أوامر SQL أبدًا بلصق مدخلات المستخدم داخل النص.
`int rows = cmd.ExecuteNonQuery();`	ExecuteNonQuery تستخدم للأوامر التي لا تعيد جدول نتائج مثل INSERT و UPDATE و DELETE، وتعيد عدد الصفوف التي تأثرت، فنحفظه في rows.
`Console.WriteLine($"تمت إضافة {rows} صف");`	نطبع رسالة تأكيد فيها عدد الصفوف المضافة، وسيكون 1 عند النجاح.

ExecuteReader للقراءة (SELECT)، وExecuteNonQuery للإضافة والتعديل والحذف، وExecuteScalar لقيمة واحدة مثل COUNT.

التمارين العملية

تمرين 1: تعديل وإحصاء

سهل

ارفع درجة كل طلاب دورة HTML بمقدار 5، ثم اعرض عدد الطلاب ومتوسط الدرجات لكل دورة.

الحل المقترح:

```SQL
UPDATE Students SET Grade = Grade + 5 WHERE Course = N'HTML';

SELECT Course, COUNT(*) AS Total, AVG(Grade) AS Avg
FROM Students GROUP BY Course;
```

ماذا يفعل هذا الكود؟ حل التمرين: زيادة درجة كل طلاب دورة HTML بمقدار 5، ثم عرض عدد الطلاب ومتوسط الدرجات لكل دورة للتأكد من التغيير.

`UPDATE Students SET Grade = Grade + 5 WHERE Course = N'HTML';`	الأمر UPDATE مع SET Grade = Grade + 5 يأخذ الدرجة الحالية لكل صف ويضيف لها 5. الشرط WHERE يحصر التعديل في طلاب دورة HTML فقط؛ وبدونه ستزيد درجات الجميع.
`SELECT Course, COUNT(*) AS Total, AVG(Grade) AS Avg`	نختار اسم الدورة، وعدد طلابها بـ COUNT(*)، ومتوسط درجاتها بـ AVG، ونعطي الأعمدة أسماء واضحة بـ AS.
`FROM Students GROUP BY Course;`	نأخذ البيانات من Students ونجمّعها بـ GROUP BY حسب الدورة، فتظهر نتيجة واحدة لكل دورة، وسترى أن متوسط HTML ارتفع بمقدار 5.

تمرين 2: عرض الطلاب من C#

متوسط

اكتب برنامج C# يتصل بـ TrainingDB ويعرض كل الطلاب الناجحين (60 فأكثر) مع دوراتهم.

الحل المقترح:

```C#
using Microsoft.Data.SqlClient;

string cs =
@"Server=localhost\SQLEXPRESS;Database=TrainingDB;Trusted_Connection=True;TrustServerCe
rtificate=True;";
using var conn = new SqlConnection(cs);
conn.Open();

var cmd = new SqlCommand("SELECT FullName, Course, Grade FROM Students WHERE Grade ≥
60", conn);
using var r = cmd.ExecuteReader();
while (r.Read())
    Console.WriteLine($"{r["FullName"]} | {r["Course"]} | {r["Grade"]}");
```

ماذا يفعل هذا الكود؟ حل التمرين: برنامج #C يتصل بقاعدة البيانات ويطبع الطلاب الناجحين (درجتهم 60 فأكثر) بالشكل: الاسم | الدورة | الدرجة.

`using Microsoft.Data.SqlClient;`	توجيه using يستورد مكتبة Microsoft.Data.SqlClient لنستخدم أصناف الاتصال بـ SQL Server مثل SqlConnection و SqlCommand.
`string cs = @"Server=localhost\SQLEXPRESS;Database=TrainingDB;Trusted_Connection=True;TrustServerCertificate=True;";`	نص الاتصال: Server اسم الخادم (نسخة SQLEXPRESS على جهازك)، و Database اسم القاعدة، و Trusted_Connection=True للدخول بحساب Windows، و TrustServerCertificate=True لقبول شهادة الخادم المحلي. علامة @ تجعل الشرطة المائلة \ حرفًا عاديًا داخل النص.
`using var conn = new SqlConnection(cs);`	ننشئ كائن الاتصال، و using var تضمن إغلاقه تلقائيًا عند الانتهاء.
`conn.Open();`	نفتح الاتصال بالخادم فعليًا.
`var cmd = new SqlCommand("SELECT FullName, Course, Grade FROM Students WHERE Grade ⩾ 60", conn);`	نجهّز أمر SELECT يجلب الاسم والدورة والدرجة للطلاب الذين درجتهم 60 أو أكثر، ونربطه بالاتصال conn.
`using var r = cmd.ExecuteReader();`	ExecuteReader تنفّذ الاستعلام وتعيد قارئًا سمّيناه r لنقرأ النتائج صفًا صفًا.
`while (r.Read())`	r.Read() تنتقل للصف التالي وتعيد true ما دام هناك صفوف، فتستمر الحلقة حتى آخر صف. لم نضع { } لأن جسم الحلقة سطر واحد فقط.
`Console.WriteLine($" {r["FullName"]} | {r["Course"]} | {r["Grade"]}");`	نطبع قيم أعمدة الصف الحالي بكتابة اسم كل عمود بين القوسين المربعين، مثل r["FullName"]، ونفصل بينها بالرمز |.

تمرين 3: تحدٍّ: عدد الطلاب ومتوسطهم من #C

تحدٍّ

استخدم ExecuteScalar لطباعة عدد الطلاب، ثم متوسط الدرجات بخانة عشرية واحدة.

الحل المقترح:

C#

```
using var conn = new SqlConnection(cs);
conn.Open();

int count = (int)new SqlCommand("SELECT COUNT(*) FROM Students", conn).ExecuteScalar();
```

```
decimal avg = (decimal)new SqlCommand("SELECT AVG(Grade) FROM Students",
conn).ExecuteScalar();

Console.WriteLine($"عدد الطلاب: {count}");
Console.WriteLine($"المتوسط: {avg:F1}");
```

ماذا يفعل هذا الكود؟ حل التمرين: برنامج يحسب عدد الطلاب ومتوسط درجاتهم من قاعدة البيانات باستخدام ExecuteScalar، ثم يطبعهما مع تقريب المتوسط لرقم عشري واحد.

`using var conn = new SqlConnection(cs);`	ننشئ اتصالًا بقاعدة البيانات باستخدام نص الاتصال cs (المعرّف سابقًا)، و using var تغلقه تلقائيًا.
`conn.Open();`	نفتح الاتصال قبل تنفيذ الأوامر.
`int count = (int)new SqlCommand("SELECT COUNT(*) FROM Students", conn).ExecuteScalar();`	ExecuteScalar تنفّذ الاستعلام وتعيد قيمة واحدة فقط (أول خانة في النتيجة)، وهي مناسبة لـ COUNT. لكنها تعيد النوع العام object، لذلك نحوّلها بـ (int) إلى رقم صحيح لنحفظها في count.
`decimal avg = (decimal)new SqlCommand("SELECT AVG(Grade) FROM Students", conn).ExecuteScalar();`	بنفس الطريقة نجلب متوسط الدرجات ونحوّله بـ (decimal) لأن العمود Grade عشري. ملاحظة: إن كان الجدول فارغًا فإن AVG تعيد NULL ويفشل هذا التحويل.
`Console.WriteLine($"عدد الطلاب: {count}");`	نطبع عدد الطلاب داخل الرسالة.
`Console.WriteLine($"المتوسط: {avg:F1}");`	نطبع المتوسط، والتنسيق F1: يعرض الرقم برقم واحد بعد الفاصلة العشرية (مثل 83.5) بدل خانات كثيرة.

اختبر نفسك (3 أسئلة)

1. ماذا يحدث عند تنفيذ DELETE FROM Students بدون WHERE؟

لا شيء | يُحذف أول صف | تُحذف كل الصفوف

2. أي دالة تحسب المتوسط؟

SUM | AVG | COUNT

3. لتنفيذ INSERT من C# نستخدم:

ExecuteReader | ExecuteNonQuery | ReadLine

الإجابات: 1) تُحذف كل الصفوف · 2) ExecuteNonQuery (3 · AVG

الواجب: أكمل المشروع النهائي وجهّز عرضًا مدته 5 دقائق.